# REQ 5 Design Rationale — Mannequin

## A. Implementation of Mannequin Behaviour

**Design 1: Implement the Mannequin's behaviour directly in its playTurn() method using boolean flags and nested if-else conditional blocks to switch between idle, active, berserk and mimic modes.**

| Pros | Cons |
|---|---|
| Simple and straightforward to implement, all logic is in one place. | All four modes are entangled in a single method. Adding a new mode requires modifying the existing conditional chain, violating the Open-Closed Principle. |
| No extra classes are needed. | The method grows large and hard to read as more modes and transitions are added. |
| | State-specific data (idle turn counter, disguise duration, display character) must all be stored as fields in Mannequin, giving Mannequin multiple responsibilities. |

**Design 2 (Chosen): Implement the State design pattern — define a MannequinState interface with evaluate(), onEnter(), and getBehaviours() methods, then create concrete state classes IdleState, ActiveState, BerserkState, and MimicState. The Mannequin delegates all behaviour to the current state object.**

| Pros | Cons |
|---|---|
| Each state class is responsible for exactly its own behaviour and transitions, separating concerns clearly (Single Responsibility Principle). | More classes are introduced, which slightly increases initial complexity. |
| Adding a new state requires only a new class that implements MannequinState, with no changes to existing states or the Mannequin itself (Open-Closed Principle). | |
| Each state's logic is self-contained and easy to test or read in isolation. | |
| Mannequin stays lightweight — it only holds a reference to the current state and delegates to it. | |

### Chosen: Design 2.

The State design pattern cleanly separates each mode's logic into its own class, making transitions explicit and the Mannequin itself easy to reason about. Each state fulfils a single responsibility and new states can be added without touching existing code.

## B. Managing State Transitions

**Design 1: Each concrete state class holds direct references to the other state objects it can transition to, constructing them at instantiation time.**

| Pros | Cons |
|---|---|
| Simple — each state knows directly what it can transition into. | Tight coupling between state classes: a change to BerserkState's constructor would require updating every state that creates it. |
| | All states are created eagerly even if never reached, wasting memory. |
| | Circular dependencies can arise (e.g., BerserkState references ActiveState, ActiveState references BerserkState). |

**Design 2 (Chosen): Introduce a dedicated StateManager class that acts as a factory for state objects. Each concrete state calls manager.berserk(), manager.active(), etc., receiving a cached instance without knowing how or when states are constructed.**

| Pros | Cons |
|---|---|
| States are decoupled from each other — no state imports another concrete state class directly (Dependency Inversion Principle). | An additional class is introduced to manage state lifecycle. |
| StateManager is the single point where state objects are created, making it easy to change construction logic in one place (Single Responsibility Principle). | |
| States are created lazily and shared via the manager, reducing object creation. | |

**Chosen: Design 2.**

The StateManager acts as a level of abstraction between concrete state classes, removing direct dependencies between them and resolving the circular reference problem. State construction is centralised and objects are reused rather than recreated.

## C. Sharing State-Specific Data Across States

**Design 1: Store all state-related data (idle turn counter, disguise turn counter, display character) directly as instance fields on the Mannequin actor, passing the actor reference to every state call.**

| Pros | Cons |
|---|---|
| No extra class needed; data is always accessible through the actor. | Mannequin accumulates fields that only apply to specific states, violating the Single Responsibility Principle. |
| | State data is exposed to the whole Mannequin class, making it possible to accidentally mutate it from outside the state machine. |

**Design 2 (Chosen): Encapsulate all state-specific data in a StateData class. The Mannequin holds one StateData instance and passes it to evaluate() and onEnter() calls so every state can read and write shared data through a single, controlled object.**

| Pros | Cons |
|---|---|
| Mannequin's own class stays clean — all mutable state-machine data is in one cohesive container (Single Responsibility Principle). | A small additional class is required. |
| All state data is accessed and mutated exclusively through StateData, preventing accidental access from unrelated code. | |
| Adding new state-shared data (e.g., a cooldown counter) only requires modifying StateData, not the Mannequin or any state class (Open-Closed Principle). | |

**Chosen: Design 2.**

Encapsulating state data in a dedicated StateData class keeps the Mannequin actor clean and ensures all mutable state-machine data is accessed through a single controlled object, reducing the risk of accidental mutation.

---

## Final Design Choice & SOLID Principles

The final design adopts Design 2 for all three decisions above. The MannequinState interface defines the contract (evaluate, onEnter, getBehaviours, getStateName) that each of the four concrete states implements. A StateManager mediates all state creation and caching, and a StateData object carries all mutable state-machine data.

This design adheres to the following SOLID principles:

- **Single Responsibility Principle:** Each state class is responsible for only its own logic and transitions; StateManager is responsible only for state construction; StateData is responsible only for holding shared data.
- **Open-Closed Principle:** New states or behaviours can be added by creating a new class implementing MannequinState with no changes to existing code.
- **Dependency Inversion Principle:** Concrete states depend on the MannequinState interface and StateManager abstraction rather than on each other's concrete classes.
- **Interface Segregation Principle:** MannequinState exposes only the methods that the Mannequin engine loop actually needs, without polluting states with unrelated methods.

BerserkState.onEnter() forces all adjacent workers to drop their entire inventory using DropAction, then AttackWorkerBehaviour and ChaseSoloWorkerBehaviour are returned as the active behaviours for that turn. MimicState.onEnter() takes the first item from the Mannequin's inventory as the disguise character, removes it, and pushes adjacent workers outward using Chebyshev distance comparison. The DecoyBehaviour then acts for 5 turns (DISGUISE_DURATION) before the state re-evaluates.